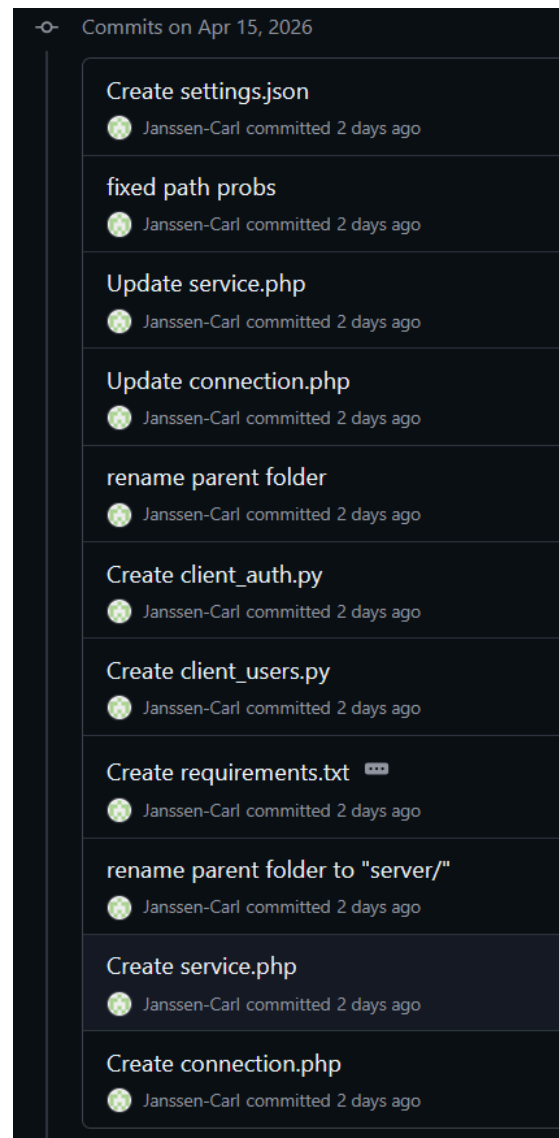


Name: Janssen Carl M. Marfil

Role: Version 1 Developer (client and server)

So what I did here was very simple. I was the one who initialized the project. I first mimicked the code from the demo and worked my way through it. The very first thing I did was to set up my environment. I initialized XAMPP, created a new database connection using MYSQL WorkBench. Once everything was configured, I then started coding the connection.php. Here I just configured it to connect to the database connection. At first I just hardcoded the database creation. I mean I already have a schema and the tables but I thought that I could've just automatically created the database schema as well as the tables if it is not yet created by adding a piece of code that does create them. (check screenshot of the code below)

One of the challenges I encountered here is thinking if I can implement a more complex api client <-> server logic. I contemplated a bit, but in the end I decided to create a more simple logic for now, just to test it out first. So I only did Auth/Login and User Management Api. But on that I have a few challenges, since this is just a console app, security is quite weak. On the Auth/Login code. First, the password was not concealed when typing it. So I checked for solutions. Fortunately, python has a lot of libraries ready to use for every problem you have. Second, my first version does not have any sort of validation before API calls. So I modified the code to have proper validation, error fallbacks,



exceptions, and I even added a confirm password prompt and email format rule. On the User Management Code. First, my first version has no deleting confirmation, so one mistyped ID can remove the wrong account. So I added a confirmation prompt. I also added confirmation on the update side, since they're basically just the same logic and they need both. I also encountered some validation issues, so I added validation measures too. As for the service part, it's quite similar, I just added validations since it lacks it. Some error handling, especially on the part of being vague on the error response, so that it wouldn't be used to verify real users. That's all of the challenges I encountered. I faced them because what I did on the first version was just bare minimum, I just wanted it to work.

Overall, I just did the simplest version of our API act, this is the V1, my groupmate also did a bit more complex version, the V2.

Figure 1. Automatic schema and table if it does not exist.

```
// Ensure schema exists, then select it.
if (!$conn->query("CREATE DATABASE IF NOT EXISTS `dbname` CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci")) {
    die(json_encode([
        "status" => "error",
        "message" => "Failed to create or verify database schema"
    ]));
}

if (!$conn->select_db($dbname)) {
    die(json_encode([
        "status" => "error",
        "message" => "Failed to select database schema"
    ]));
}

// Ensure users table exists.
$createUsersTableSql = "
    CREATE TABLE IF NOT EXISTS users (
        id INT AUTO_INCREMENT PRIMARY KEY,
        username VARCHAR(100) NOT NULL,
        email VARCHAR(150) NOT NULL UNIQUE,
        password VARCHAR(255) NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
";

if (!$conn->query($createUsersTableSql)) {
    die(json_encode([
        "status" => "error",
        "message" => "Failed to create or verify users table"
    ]));
}
```

Go to Line/Column